



TELECOMPLUS REPORT

Group Number: 022

Group Members:

- **Gabriel Calero -20240357**
- **Malek Ben Hamza -20240356**
- **Bouthaina Hammami - 20240411**
- **Hanen Salhi - 20240355**



TABLE OF CONTENTS

1.	Business Presentation	4
2.	Business problem	6
3.	Business needs.....	7
3.1.	Improving Service Quality and Customer Experience	7
3.2.	Customer Preferences and Product Trends	7
3.3.	Revenue Analysis and Market Performance.....	8
4.	Overview of TelecomPlus data structure	8
4.1.	Channel Info	8
4.2.	Interaction Reason Info	9
4.3.	Customer Info	9
4.4.	Agent Info	10
4.5.	Plans Info	10
4.6.	Customer Interactions Cases	11
4.7.	Signups Record.....	11
4.8.	Geographic Coordinates of U.S. Cities (external source)	11
5.	Data Warehouse design	12
5.1.	Data Warehouse presentation and design methodology	12
5.1.1.	Business requirements gathering	12
5.1.2.	Identifying Key Business Processes and Fact Tables	12
5.1.3.	Defining dimension tables	13
5.1.4.	Star schema design	13
5.2.	Dimension tables.....	14
5.2.1.	Dim_date table	14
5.2.2.	Dim_customer table	16
5.2.3.	Dim_agent table.....	17
5.2.4.	Dim_channel table.....	18
5.2.5.	Dim_plan table.....	19
5.2.6.	Dim_reason table	20
5.3	Fact tables	21

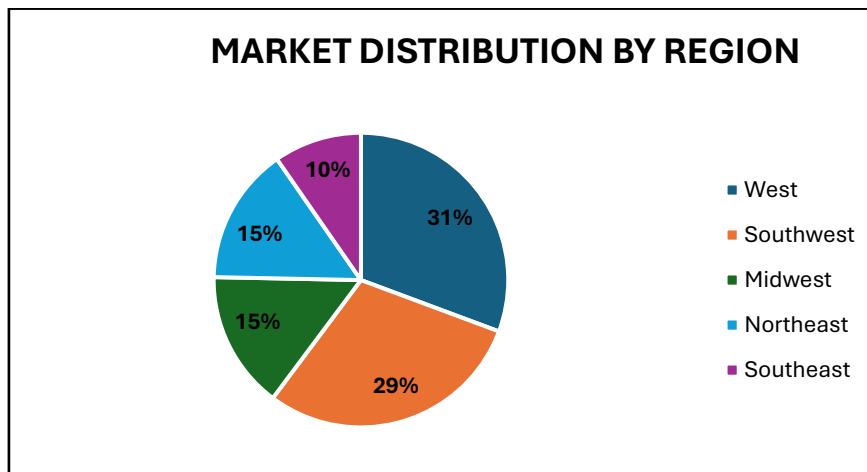
5.3.1. fact_customer_interactions_cases.....	21
5.3.2. fact_signups_record.....	23
5.4. Data Warehouse Schema	23
6. ETL Process.....	25
6.1. Overview of the Staging Area workflow	25
6.2. Loading Dimension Tables	26
6.2.1. Stg_dim_customer.....	26
6.2.2. Stg_dim_agent	26
6.2.3. Stg_dim_date.....	29
6.2.4. Stg dim_reason	29
6.2.5. stg_dim_channel	30
6.2.6. stg_dim_plan	30
6.3. Loading Fact tables	30
6.3.1. Stg_fact_customer_interactions_cases	30
6.3.2. Stg_fact_signups_record	31
6.4. Quality Check Validation	33
6.5. Data Loading Process into the Data Warehouse	34
7. Extra ETL work	35
7.1. Incremental load	35
7.1.1. Changes to the staging area pipeline and dataflow	35
7.1.2. Modifications to the Data Warehouse Loading Pipeline	38
7.2. Slowly changing dimension.....	39
8. Lessons Learned	41
9. Conclusion.....	41

1. Business Presentation

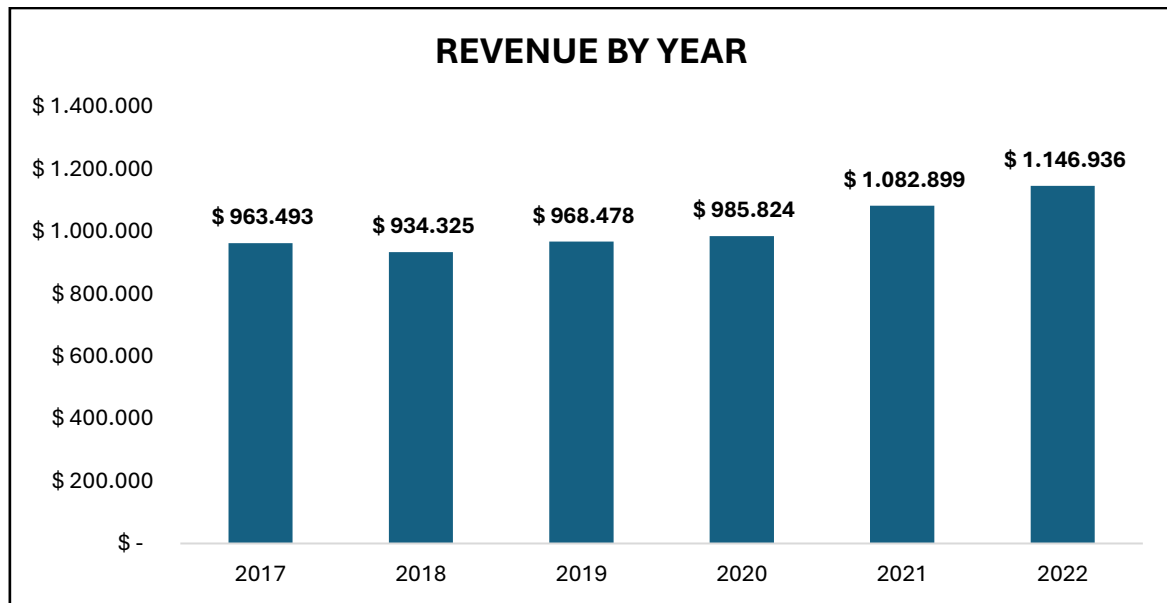
TelecomPlus is a telecommunications company that delivers mobile and internet services to millions of people across the United States. The company offers a variety of subscription plans tailored to different needs, including prepaid and postpaid mobile options, as well as high-speed fiber internet for both residential and business use. **TelecomPlus** is committed to providing reliable and flexible plans, making it easy and accessible for everyone to stay connected—whether for individual customers or small businesses.

MOBILE INTERNET		BASIC PREPAID PLAN	5 GB
		PREMIUM PREPAID PLAN	15 GB
		UNLIMITED POSTPAID PLAN	UNLIMITED!
HOME & BUSINESS INTERNET		HOME FIBER PLAN	speed: 100 Mbps
		ULTRA FIBER PLAN	speed: 300 Mbps
		ENTERPRISE BUSINESS PLAN	SPEED: 500 MBPS!

As of today, we have a market presence across the entire country. The West region (including the West, Southwest, and Midwest areas) accounts for most of our customer base, representing 75% of the total. However, there is still significant growth potential in the East region (Northeast and Southeast), where the remaining 25% of our customers are located.



In terms of revenue, the company has experienced an average annual growth of 5% over the past five years, with the biggest increase happening between 2020 and 2021, when it grew by 12%. However, during recent board meetings, shareholders have expressed dissatisfaction with this growth rate, as they feel it is not large enough to ensure the company's profitability in the long term.



To address this, TelecomPlus has strengthened its customer service processes as part of its five-year strategic plan. The goal is to offer closer and more effective support to help customers resolve technical or administrative issues related to our services. Additionally, we aim to use these customer interactions as an opportunity to attract potential new clients and expand our annual revenue base.

Taking advantage of the rise of digital platforms, we have developed a variety of communication channels to meet the needs of our different market segments, based on their demographics. These channels include remote options, such as digital and phone support, as well as in-person options through our physical stores in New York, Chicago, and Los Angeles.

CHANNEL TYPE	PLATFORM
Digital	Support Email
	Sales Email
	Marketing Email
	Website Chat
	Mobile App Chat
	Facebook Messenger
	Twitter
	Facebook
	Instagram
Voice	Toll-Free Line
	Local Line
	Customer Support Line
In-Person	New York Branch
	Los Angeles Branch

CHANNEL TYPE	PLATFORM
	Chicago Branch

The growing competition in the industry makes it essential for TelecomPlus to better understand its revenue streams, customer preferences (especially in regions where our market share is low), and the quality of interactions with the public. This is crucial for improving customer loyalty and overall satisfaction, both before and after the sale of our services.

The objective of this project is to implement a Business Intelligence solution that will help the company gain a detailed understanding of the factors that have the greatest impact on sales and customer service. This will support better decision-making for future strategies and business growth.

2. Business problem

TelecomPlus is dealing with a few big challenges that are holding it back from giving the best service, meeting local customer needs, and growing its revenue.

- **Inconsistent Service Quality Across Channels**

TelecomPlus lacks a clear understanding of the performance of our communication channels—such as phone, email, chat, and social media—within our customer service area. This makes it difficult to find bottlenecks in customer service, such as excessive wait times and prolonged resolution times. We are committed to providing a better service for our customers.

- **Products That Don't Fit Local Needs**

Customers in different areas have different needs, but **TelecomPlus** doesn't fully understand these regional preferences. This means the company might be offering products that don't fit what local customers really want, missing out on chances to connect with them and build loyalty.

- **Unclear Revenue Sources and Growth Opportunities**

TelecomPlus isn't sure which locations and products are bringing in the most revenue. Without insights into which markets and product lines are performing best, the company risks investing time and money in the wrong areas, missing out on growth opportunities.

3. Business needs

3.1. Improving Service Quality and Customer Experience

Goal: To gain a clear understanding of how each customer service channel and department perform.

- **What is the average waiting and resolution time for each department?**

Understanding the average waiting and resolution time for each department helps identify bottlenecks in customer service and highlights areas where efficiency can be improved. If certain departments consistently have higher wait or resolution times, it can lead to customer dissatisfaction and churn.

- **What is the average customer satisfaction score for each channel and department?**

It identifies which channels and departments meet customer expectations and highlights areas needing improvement to boost satisfaction and loyalty. High satisfaction levels can indicate effective processes, while low levels can point to specific channels or departments needing improvement.

- **Which channel resolves interactions the fastest for specific reasons of interaction?**

Understanding which channel resolves interactions the fastest helps optimize customer service efficiency, reduce wait times, and enhance overall customer satisfaction.

- **What is the volume of interactions handled by each channel for different reasons of interaction?**

Understanding the volume of interactions handled by each channel for different reasons of interaction helps identify demand patterns. It allows the company to optimize staffing, prioritize high-demand channels, and improve service delivery.

3.2. Customer Preferences and Product Trends

Goal: To understand the preferences of customers in different areas so that we can deliver the right products to the right audience.

- **Which products have the highest number of subscriptions, broken down by gender and age group?**

Understanding how product preferences differ by gender and age helps tailor marketing strategies, product offerings, and promotions to better meet the needs of specific customer segments, driving higher engagement and sales.

- **What are the top 3 products with the highest number of subscriptions by region?**

Identifying the top products by new subscriptions in each region helps the company understand regional preferences, optimize marketing strategies, and focus on high-demand products to drive growth in specific areas.

3.3. Revenue Analysis and Market Performance

Goal: To analyze revenue patterns across locations, products, and time to help TelecomPlus spot growth opportunities and focus on areas that drive profitability.

- **What are the top 5 locations with the highest revenue?**

This question looks at which areas bring in the most revenue, giving TelecomPlus a sense of its strongest markets. Knowing where the top-performing regions are can guide investment and help the company focus on high-potential areas.

- **How has the annual revenue for each product category changed over time?**

This question focuses on revenue trends over the years for different product categories, like mobile or (fixed) internet plans. By tracking these trends, TelecomPlus can see which categories are growing and which might need a boost, helping with decisions on product development and promotion.

- **Which product types within categories have shown the most significant revenue growth or decline over the years?**

This question goes into the details of which specific product types are gaining or losing traction. It allows TelecomPlus to focus on expanding the popular products and reconsidering those that may be underperforming, helping the company stay aligned with customer demand and market trends.

4. Overview of TelecomPlus data structure

This dataset gives a full picture of how **TelecomPlus** connects with its customers, from the plans they subscribe to and the channels they use to reach out, to details about who their customers are and why they're contacting them. It includes everything from customer demographics and agent roles to satisfaction scores and subscription timelines. Altogether, these files create a well-organized dataset that helps **TelecomPlus** understand customer behavior, improve service quality, and fine-tune its product offerings.

4.1. Channel Info

Lists of the various channel platforms through which customers interact with the company, organized hierarchically by type based on their nature (Email, Chat, Phone, Store Visit, Social

Media). Each type is subordinated to a main category (Digital, Voice, In-Person), providing a clear structure for understanding how customer interactions are classified.

Channel_info	
channel_id	Individual identifier for the communication channel
channel_category	Channel category (digital, in-person, voice)
channel_type	Types of each category of channel. It can be (email, chat, phone, store visit, social media)
channel_Platform	Channel platform that a customer may use to reach an agent

4.2. Reason Info

Captures the two main reasons why customers contact **TelecomPlus**, such as signing up for a service or requesting support. It helps us understand the various needs that bring customers to the company.

Interaction_reason_info	
reason_id	Individual identifier of the reason for the interaction
reason_name	Indicates the reason of customer's call (to sign up or asking for support)
category	Groups reasons categories (e.g., "Inquiry," "Assistance").
type	Specifies the nature or type of interaction within a category (e.g., "Sales/Onboarding," "Technical Support").
Call_duration	Indicates the urgency level of the interaction, enabling prioritization in reporting (e.g., "High," "Medium").

4.3. Customer Info

It provides demographic and location information about each customer, including attributes such as age, gender, city, state, and region, giving **TelecomPlus** a better understanding of who its customers are.

Customer_info	
customer_id	Individual identifier for the customer
first_name	Customer's first name
surname	Customer's surname
age	Customer's age

Customer_info	
gender	Customer's gender (male/female)
city	Customer's city
state	Customer's state

4.4. Agent Info

Contains details about **TelecomPlus** agents, including their names, roles, teams, and departments in a hierarchical structure. It illustrates how each agent is organized within the company, with departments overseeing teams, and teams comprising specific roles.

Agent_info	
agent_id	Individual identifier for each agent
name	Agent's name
agent_role	The role of each agent
Agent_team	Agents are divided into teams
Agent_department	Departments every team works in (sales, customer service, technical support)
hire_date	It is the date an agent officially starts working for the company.

4.5. Plans Info

Details of the subscription plans offered by TelecomPlus, which are categorized into offer types such as Prepaid, Postpaid, Fiber, and Business Internet. Each of these offer types is, in turn, subordinated to broader categories of Mobile and (Fixed) Internet.

Plans_info	
offer_category	Main category (mobile, fixed Internet)
offer_type	Type of the product (prepaid, postpaid, fiber, business)
product_name	Name of the product
monthly_price	Monthly price of the plan
data_allowance	Maximum amount of data included (only for mobile category plans)
internet_speed	Internet speed included (only for fixed internet category plans)
description	Description of the plan
plan_id	Unique identifier for the plan

4.6. Customer Interactions Cases

Logs each customer interaction, including wait times, resolution times, and satisfaction scores. This data is critical for analyzing service quality and customer satisfaction.

customer_interactions_cases	
customer_id	Individual identifier for the customer
agent_id	Individual identifier for the agent
interaction_date	Date of the interaction
interaction_time	Time of the interaction
reason_id	Individual identifier for the reason of the interaction
channel_id	Individual identifier for the communication channel
wait_time	Customer's waiting time until he gets a response from an agent (in minutes)
resolution_time	The amount of time taken to resolve a customer's issue or need (in minutes)
satisfaction_score	Satisfaction level of customers after interacting with an agent

4.7. Signups Record

Records information on customer subscriptions, including discounts, final prices, and the start and end dates of each subscription. It gives **TelecomPlus** insight into customer purchase behavior and helps analyze the lifecycle of subscriptions.

signups_record	
customer_id	Individual identifier for the client
plan_id	Individual identifier for the plan
percentage_discount	Discount percentage applied
signup_date	Start date of the subscription
termination_date	Date of cancelling the subscription

4.8. Geographic Coordinates of U.S. Cities (external source)

This dataset contains the geographic coordinates (latitude and longitude) for cities across the United States, sourced from an [external provider](#).

uscoordinates_info	
lat_info	Represents the latitude of a city, indicating its north-south position on the Earth's surface.
lng_info	Represents the longitude of a city, indicating its east-west position on the Earth's surface.
city_name	The name of the city.
state_id	A unique identifier for the state in which the city is located.
state_name	The name of the state in which the city is located.
Region_name	The name of the region which the city is located

5. Data Warehouse design

5.1. Data Warehouse presentation and design methodology

To address TelecomPlus's business needs and to solve the main business information problems, we move on to the design and implementation of the data warehouse. The methodology used was the Kimball dimensional modeling approach, focusing on creating a star schema that supports easy analysis of key metrics and trends.

5.1.1. Business requirements gathering

The data warehouse was designed to meet the company's goals of improving service quality, understanding customer preferences, and analyzing revenue performance by answering key business questions.

5.1.2. Identifying Key Business Processes and Fact Tables

Based on the business information problem and the specific business needs, the core processes identified for the analysis were **customer interactions** and **subscriptions to products**.

Taking that into consideration, the fact tables were defined focused on capturing data from two key areas: **general support interactions**, where clients seek assistance, and **sales**, where clients communicate to subscribe to the company's products.

Table name	Description
fact_customer_interactions_cases	This table records client interactions across the company's communication channels. It captures details such as the attending agent, customer information, the purpose of the interaction (e.g., support or product subscription), wait and

Table name	Description
	resolution times, customer satisfaction scores, satisfaction levels (categorical), and interaction duration.
fact_signups_record	This table captures data on new product subscriptions offered by the company. It includes details such as the subscription's sign-up date, termination date, customer information, the type of plan selected, any discounts applied to the subscription (if applicable), and the final price paid. This data provides a view of the company's subscription activity and customer preferences.

5.1.3. Defining dimension tables

Based on our chosen model, the dimension tables are the tables that surround and are directly linked to the fact tables to provide context to the analysis. In that sense, the following tables were defined to facilitate meeting the specific business information needs:

Table name	Description
dim_date	This table covers all the values of dates (between 2017 and 2022) with granularity of day, month, quarter, semester and year
dim_customer	This table contains customer data (first and surname, age, gender, location)
dim_agent	This table includes agent informations (name, role, team, department and experience years)
dim_channel	This table stores data regarding the channel platforms through which customers interact with the company
dim_plan	This table covers all the details of the subscription plans offered by TelecomPlus such as product name, offer category, offer type, monthly price and allowed data)
dim_reason	This table contains the reason for the interaction (reason 1: either the client requires some kind of support or reason 2: desires to sign up for one of the products the company offers).

5.1.4. Star schema design

The selected schema connects the **fact_customer_interactions_cases** and **fact_signups_record** tables with the dimensions, enabling the company to analyze:

- Customer service performance across channels and departments.
- Product subscription trends by demographics, region, and time.
- Revenue patterns across locations, products, and time.

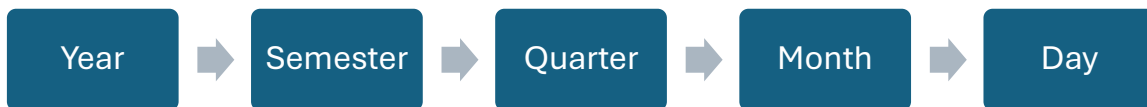
5.2. Dimension tables

The following is a more detailed view of the dimension tables, including their structure, the type of data they contain, and their relationship to the specific business needs.

5.2.1. Dim_date table

The date dimension enables analysis of revenue trends by providing time-based attributes such as year, quarter, and month. This allows the company **to track annual revenue changes for product categories** and **identify growth or decline patterns in specific product types over time**.

Regarding hierarchies, the date dimension table is disaggregated into levels such as year, semester, quarter, month, and day.



Column name	Data type	Description
sk_date	INT	Surrogate key
proper_date	DATE	Date in format "dd/mm/yyyy"
monthday_number	INT	Month number
weekday_number	INT	Number of the weekday
weekday_name	VARCHAR(50)	Name of the weekday

Column name	Data type	Description
weekday_name_short	VARCHAR(50)	Shorter version of the name of the day
weekday_type	VARCHAR(10)	Conditional column that takes values of either 'Weekday' or 'Weekend' depending on the day of the week.
is_special	VARCHAR(30)	Conditional column if the date is a holiday
month_number	INT	Number of the month [1, 12]
month_name	VARCHAR(50)	Name of the month
month_name_short	VARCHAR(50)	Shorter version of the name of the month
quarter_number	INT	Number of the quarter [1, 4]
quarter_name_short	VARCHAR(10)	Shorter version of the name of the quarter
quarter_name	VARCHAR(50)	Name of the quarter
semester_number	INT	Number of the semester [1, 2]
semester_name_short	VARCHAR(10)	Shorter version of the name of the semester
semester_name	VARCHAR(50)	Name of the semester
year	INT	Year corresponding to the date
Is_weekend	Varchar(3)	"yes" If the day in on weekend,"No" if so
Season_number	INT	Number of the season [1,2,3,4]
Season_name	Varchar(10)	name of the season

Column name	Data type	Description
Season_name_short	Varchar(10)	Shorter version of the name of the season
Promotion_seasonality	Varchar(50)	Refers to the specific seasonal events or periods during which a promotion is active or targeted

5.2.2. Dim_customer table

The customer dimension table contains customer-related data such as customer ID, gender, age group, and location. It helps **segment and analyze subscription data by demographic factors**, allowing insights into product preferences by gender, age, and region, **as well as identifying top-performing locations based on revenue**.

The customer dimension table organizes the **location hierarchy** of the client, divided into **region, state, and city**. This hierarchical structure enables detailed geographic analysis of customer data.



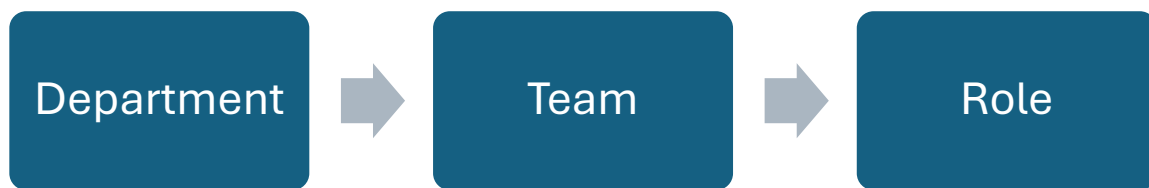
Column name	Data type	Description
sk_customer	INT	Surrogate key
bk_customer	VARCHAR(10)	Business key of the customer

Column name	Data type	Description
customer_first_name	VARCHAR(50)	Customer's first name
customer_surname	VARCHAR(50)	Customer's surname
customer_age	INT	Customer's age
customer_gender	VARCHAR(10)	Customer's gender
customer_city	VARCHAR(50)	Customer's city of residence
customer_region	VARCHAR(50)	Region within the country where the city is located
customer_state	VARCHAR(50)	Customer's state of residence
customer_city_longitude	DECIMAL(18, 2)	The geographical coordinate specifying the north-south position
customer_city_latitude	DECIMAL(18, 2)	The geographical coordinate specifying the east-west position
Load_date_time	DATETIME2(2)	The specific date and time when data is imported or loaded into DW

5.2.3. Dim_agent table

The agent dimension table contains data about agents, including agent ID, role, team, and department. **It helps analyze waiting and resolution times, as well as customer satisfaction, by breaking down data by agent roles and teams within departments.**

This dimension organizes the hierarchy of agents, detailing each agent's **role**, which is part of a **team**, and each team belongs to a **department**. This structure supports analysis of performance and efficiency across organizational levels.



Column name	Data type	Description
sk_agent	INT	Surrogate key
bk_agent	VARCHAR(10)	Business key of the agent
agent_name	VARCHAR(100)	Agent's full name
agent_role	VARCHAR(50)	Agent's role in a team
agent_team	VARCHAR(50)	Agent's teams within a department
agent_department	VARCHAR(50)	Agent's department
agent_experience_years	INT	Agent's years of experience
agent_experience_LVL	VARCHAR(50)	Agent's experience level depending on the number of years.

5.2.4. Dim_channel table

A channel dimension table contains data about customer interaction channels, including channel type, category and platform. It **helps analyze customer satisfaction, resolution times, and interaction volumes across different channels**, optimizing service efficiency and resource allocation.

The channel dimension table is structured with three levels: **category**, **type**, and **platform**. For example, "Digital" is the category, "Email" is the type, and "Support Email" is the platform.



Column name	Data type	Description
sk_channel	INT	Surrogate key
bk_channel	VARCHAR (10)	Business key of the channel
channel_category	VARCHAR (50)	Channel's main category
channel_type	VARCHAR (50)	Channel's type within a category
channel_platform	VARCHAR (100)	Channel's specific platform depending on the type

5.2.5. Dim_plan table

The plan dimension table contains details about the company's products (plans), such as product name, product category, product type, and pricing details. **It supports analysis of revenue trends, growth or decline in product performance**, helping optimize product offerings and marketing strategies.

The plan dimension table follows a hierarchy of three levels: **category**, **type**, and **product**. For example, "Mobile" is the category, "Prepaid" is the type, and "Basic Prepaid Plan" is the product.



Column name	Data type	Description
sk_plan	INT	Surrogate key
bk_plan	VARCHAR (10)	Business key of the offered plan
product_name	VARCHAR (100)	Name of the product offered
offer_category	VARCHAR (50)	Product's category
offer_type	VARCHAR (50)	Type in which the category is immersed
monthly_price	DECIMAL (18, 2)	Product's monthly price
data_allowance	VARCHAR (20)	Total data allowance (in GB)

5.2.6. Dim_reason table

The reason dimension table contains the reasons for customer interactions, such as general support or plan subscription. It **helps differentiate and analyze interactions by purpose, providing insights into the nature of customer needs** and guiding resource allocation.

Column name	Data type	Description
sk_reason	INT	Surrogate key
bk_reason	VARCHAR (10)	Business key of the reason
reason_name	VARCHAR (50)	Name of the reason for the interaction with a customer.
category	VARCHAR (50)	Groups reasons categories (e.g.,

Column name	Data type	Description
		"Inquiry," "Assistance").
type	VARCHAR (50)	Specifies the nature or type of interaction within a category (e.g., "Sales/Onboarding," "Technical Support").
Call_priority	VARCHAR (10)	Indicates the urgency level of the interaction, enabling prioritization in reporting (e.g., "High," "Medium").

5.3 Fact tables

The following is a more detailed view of the fact tables, including their structure, the type of data they contain, and, in the case of the measures, the corresponding business need to which they are addressing.

5.3.1. fact_customer_interactions_cases

This fact table tracks customer interactions with the company across various channels. It stores key metrics such as wait time, resolution time, satisfaction score, satisfaction level, and interaction duration.

Column name	Data type	Description	Related business need
fk_date	INT	Foreign key referencing dim_date (sk_date)	-
fk_agent	INT	Foreign key referencing dim_agent (sk_agent)	-

Column name	Data type	Description	Related business need
fk_channel	INT	Foreign key referencing dim_channel (sk_channel)	-
fk_customer	INT	Foreign key referencing dim_customer (sk_customer)	-
fk_reason	INT	Foreign key referencing dim_reason (sk_reason)	-
wait_time	DECIMAL (18, 2)	The amount of time a customer waits before being attended to by an agent.	What is the average waiting and resolution time for each department?
resolution_time	DECIMAL (18, 2)	The time taken to resolve a customer's issue or complete their request.	- What is the average waiting and resolution time for each department? - Which channel resolves interactions the fastest for specific reasons of interaction?
satisfaction_score	INT	A numerical rating provided by the customer to indicate their level of satisfaction with the interaction.	What is the average customer satisfaction score for each channel and department?
satisfaction_LVL	VARCHAR (50)	A categorical classification of customer satisfaction.	-
interaction_duration	DECIMAL (18, 2)	The total length of time spent on customer interaction.	- What is the average waiting and resolution time for each department? - Which channel resolves interactions the fastest for specific reasons of interaction?

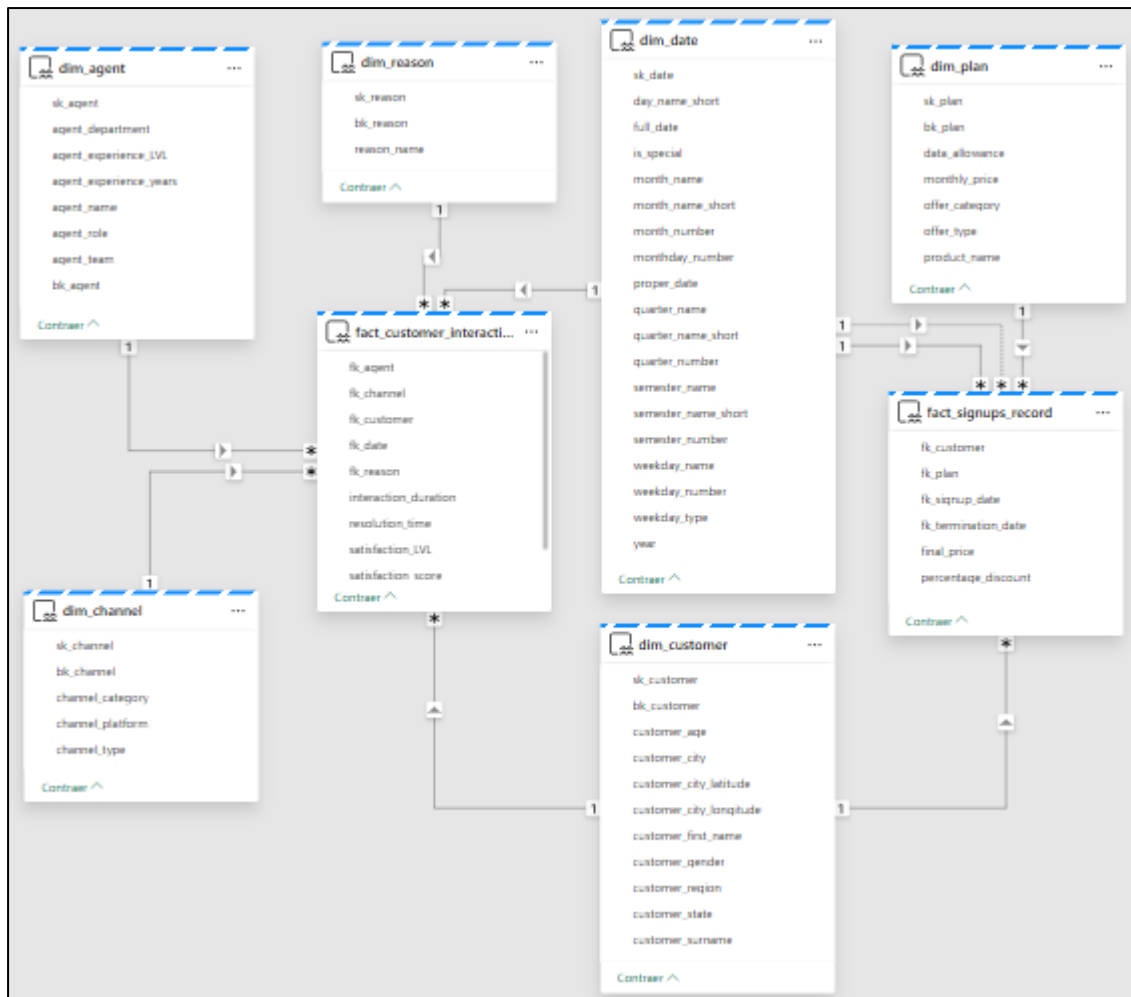
5.3.2. fact_signups_record

This fact table records information about customer subscriptions to the company's products. It captures the signup date, termination date (if applicable), customer details, plan information, applied discount, and the final price of the subscription.

Column name	Data type	Description	Related business need
fk_signup_date	INT	Foreign key referencing dim_date (sk_date)	-
fk_termination_date	INT	Foreign key referencing dim_agent (sk_agent)	-
fk_customer	INT	Foreign key referencing dim_channel (sk_channel)	-
fk_plan	INT	Foreign key referencing dim_customer (sk_customer)	-
percentage_discount	DECIMAL (18, 4)	Represents the percentage discount applied to a product	• What are the top 5 locations with the highest revenue? • How has the annual revenue for each product category changed over time? • Which product types within categories have shown the most significant revenue growth or decline over the years?
final_price	DECIMAL (18, 4)	The final price of the product or service after any discounts	

5.4. Data Warehouse Schema

Moving to the modeling phase, which involves designing the logical and physical structure of a data warehouse, we opt for the star schema following Kimball's guidelines. This schema is



known as the simplest and most common model. It is characterized by one or more central fact tables directly linked to multiple dimension tables.

To better understand the relationship between dimension tables and fact tables, here is a bus matrix that summarizes the connections between fact and dimension tables:

Fact table / Dimension table	Dim_date	Dim_customer	Dim_agent	Dim_channel	Dim_plan	Dim_reason
Fact_customer_interactions_case	X	X	X	X		X
Fact_signups_record	X	X			X	

The following is the complete database diagram for the data warehouse:

As shown in Figure 9, the fact tables `fact_customer_interaction_cases` and `fact_signups_record` are linked with the different dimension tables using the corresponding foreign keys. The fact table **`fact_customer_interaction_cases`** is connected to five dim tables:

dim_agent, dim_channel, dim_customer, dim_date, dim_reason; while the fact table **fact_signups_record** is connected to three dim tables: dim_plan, dim_date, dim_customer.

6. ETL Process

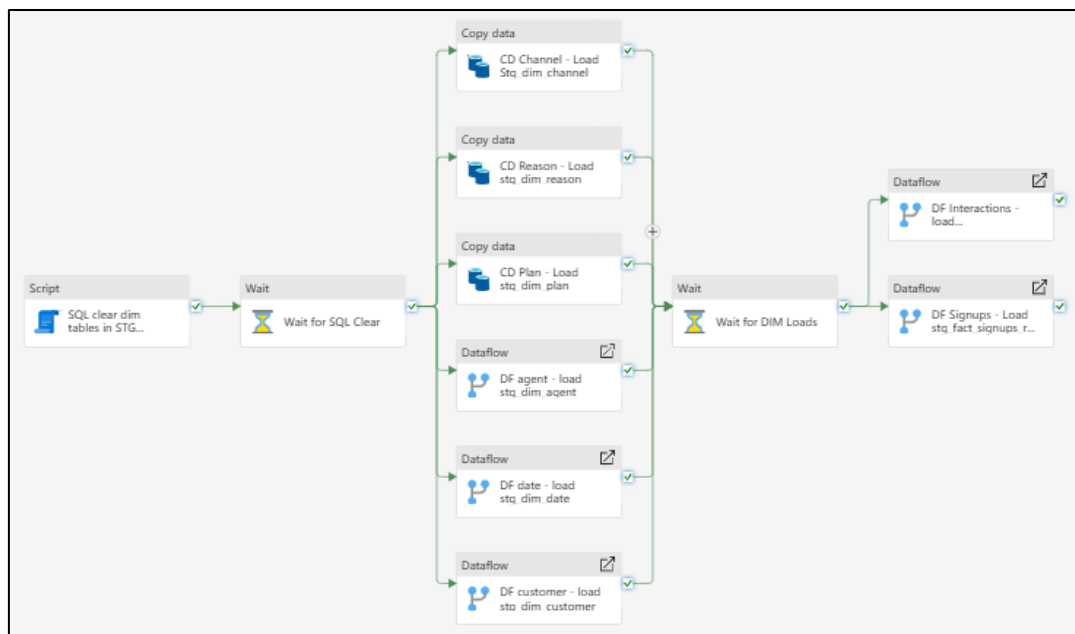
After designing the data warehouse, in a way that can answer the business questions, we need to start the ETL process that consists of extracting the data from its source, applying the necessary transformations and loading it into the data warehouse. To do so, we should first create the staging area where the ETL process will be performed.

6.1. Overview of the Staging Area workflow

The staging area is a temporary area of the architecture of a data warehouse, where raw data from various sources is brought together, cleaned and prepared for further processing into the data warehouse.

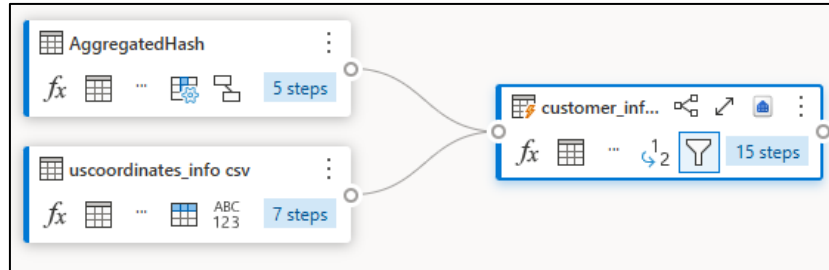
The pipeline for the staging area utilizes a mixed approach between full and incremental load. The incremental load is implemented in the two fact tables and the **stg_dim_customer** table and will be explained in more detail later.

The workflow begins by clearing all the DIM tables except for the **stg_dim_customer** table. It then proceeds with a combination of **CopyData** activities and **DataFlows Gen2** for the ETL process, pulling data from the sources in the LakeHouse and Staging Area. Finally, after loading all the DIM tables, two additional dataflows are used to load the FACT tables.



6.2. Loading Dimension Tables

6.2.1. Stg_dim_customer



This transformation involves three main queries. However, the **AggregatedHash** query will be explained in more detail in the additional ETL work, as it is part of the slowly changing dimension architecture of the table.

Merge ⓘ

Select a table and matching columns to create a merged table.

customer_info csv

customer_id	age	gender	city	first_name	surname	state
C90963	67	male	Columbus	John	Brown	Ohio
C140041	57	female	Philadelphia	Laura	Garcia	Pennsylvania
C379062	61	female	Denver	Laura	Brown	Colorado
C72349	72	female	San Jose	Emily	Davis	California

Right table for merge *

uscoordinates_info csv

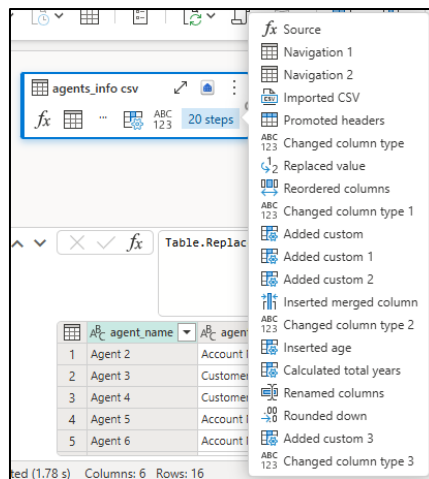
lat_info	lng_info	city_name	state_id	state_name	region_name
18.18027	-66.75266	Adjuntas	PR	Puerto Rico	Caribbean (Territory)
18.36075	-67.17541	Aguada	PR	Puerto Rico	Caribbean (Territory)
18.45711	-67.12732	Arecibo	PR	Puerto Rico	Caribbean (Territory)

✓ The selection matches 10,000 of 10,000 rows from the first table

The transformation process for the **stg_dim_customer** table included a key operation of **data enrichment** through a left outer join using the **Merge Queries** function. The **city** column from the customer table was matched with the **city_name** column from the **uscoordinates_info.csv** file. This added geographic information, including **lat_info** (latitude), **lng_info** (longitude), and **region_name** (region or state).

6.2.2. Stg_dim_agent

The transformation for the table was made in a Dataflow Gen 2 in a single query comprised of 20 steps:



The table is sourced from the 'agents_info.csv' file in the LakeHouse. The main challenge was handling the date format, which required creating new columns to extract the year, month, and day. These elements were then combined into a single new column.

hire_date	new_hire_date
31032018	3/31/2018
30102019	10/30/2019
20200113	1/13/2020
20180614	6/14/2018

The custom columns were created to extract specific date components from the original **hire_date** string, which was formatted either as YYYYMMDD or DDMMYYYY. The **hire_year**, **hire_month**, and **hire_day** columns were derived from the text values through a combination of checks and string manipulations. These components were then combined using a '/' delimiter to create a new column, **new_hire_date**, which represents the date in a standardized format.

```

hire_year
Custom column formula *
= if Text.Contains(Text.From([hire_date]), "202")
then Text.Mid(Text.From([hire_date]), Text.
PositionOf(Text.From([hire_date]), "202"), 4)
else if Text.Contains(Text.From([hire_date]),
"201") then Text.Mid(Text.From([hire_date]),
Text.PositionOf(Text.From([hire_date]), "201"), 4)
else null

```

This code adds a new column, **hire_year**, which extracts the year from a **hire_date** column containing 8-digit strings, such as "31032018". The pattern in the string represents a date in the format DDMMYYYY, where the last four digits correspond to the year. The logic checks for years starting with "202" or "201", as these represent valid years, and extracts the 4-digit year portion. If no valid year is found, the value is set to null.

hire_month

Custom column formula * ⓘ

```
= if Text.Middle(Text.From([hire_date]), 0, 4) = [hire_year] then Text.Middle(Text.From([hire_date]), 4, 2)
    else if Text.Middle(Text.From([hire_date]), 4, 4) = [hire_year] then Text.Middle(Text.From([hire_date]), 2, 2)
    else null
```

This code checks whether the **hire_year** appears in the first four digits (YYYYMMDD format) or the last four digits (DDMMYYYY format) of the **hire_date** string. Based on its position, it extracts the two digits representing the **hire_month**. If neither pattern matches, it assigns null.

hire_day

Custom column formula * ⓘ

```
= if Text.Middle(Text.From([hire_date]), 0, 4) = [hire_year] then Text.Middle(Text.From([hire_date]), 6, 2)
    else if Text.Middle(Text.From([hire_date]), 4, 4) = [hire_year] then Text.Middle(Text.From([hire_date]), 0, 2)
    else null
```

This code extracts the **hire_day** from the **hire_date** string (formatted as either YYYYMMDD or DDMMYYYY). It checks if the **hire_year** is in the first four digits (YYYYMMDD format) or the last four digits (DDMMYYYY format). If the year is in the first four digits, it extracts the last two digits as the **hire_day**. If the year is in the last four digits, it extracts the first two digits as the **hire_day**. If neither condition matches, it assigns null.

Merge columns

Choose how to merge the selected columns.

Separator *

--Custom--

/

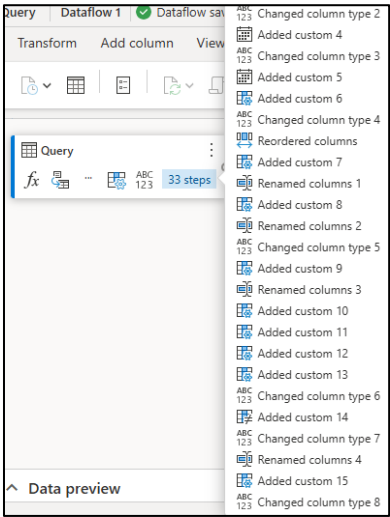
New column name

new_hire_date

Finally, the **Merge Columns** option was used with a custom delimiter ('/') to combine the extracted components into a new column named **new_hire_date**.

Other relevant transformations included calculating the **age**, which represents the time elapsed between an employee's hire date and the current date, as well as categorizing this time into levels in the **agent_experience_LVL** column.

6.2.3. Stg_dim_date



The **stg_dim_date** table is programmatically generated within the Microsoft Fabric environment using a range of dates. The **proper_date** field consists of sequential dates starting from January 1, 2017, to the current date, while **sk_date** is a surrogate key formatted as YYYYMMDD. The transformation process involves several steps.

First, the **List.Dates** function generates the date range. Next, column types are standardized, ensuring that fields like **proper_date** use the Date type, while other fields are Text or Integer. Derived columns are added to enhance functionality, including numeric and textual representations of the day of the week, quarter details, among others. Columns are then reordered for clarity and optimal loading into the final structure. Finally, the data undergoes validation to ensure date fields and derived values are accurate and aligned correctly.

6.2.4. Stg dim_reason

The **stg_dim_date** table was created using a Copy Data activity in the pipeline, with data sourced from the **reason_info.csv** file hosted on the Lakehouse. No additional transformations were required during this process.

☐	reason_id	abc string	→	bk_reason	ANY varchar
☐	reason_name	abc string	→	reason_name	ANY varchar
☐	category	abc string	→	category	ANY varchar
☐	type	abc string	→	type	ANY varchar
☐	call_priority	abc string	→	call_priority	ANY varchar

6.2.5. stg_dim_channel

The **stg_dim_channel** table was created using a Copy Data activity in the pipeline, with data sourced from the **channel_info.csv** file hosted on the Lakehouse. The data was transferred as-is, with no additional transformations required during the process.

☐	channel_id	abc string	→	bk_channel	ANY varchar
☐	channel_category	abc string	→	channel_category	ANY varchar
☐	channel_type	abc string	→	channel_type	ANY varchar
☐	channel_platform	abc string	→	channel_platform	ANY varchar

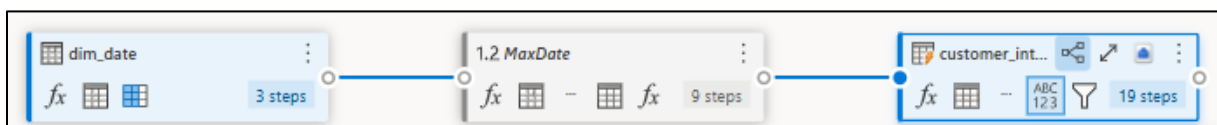
6.2.6. stg_dim_plan

The **stg_dim_plan** table was generated through a Copy Data activity in the pipeline, leveraging data from the 'plans_info.csv' file stored in the Lakehouse. This process involved directly copying the data without performing any additional transformations, ensuring the source structure remained intact.

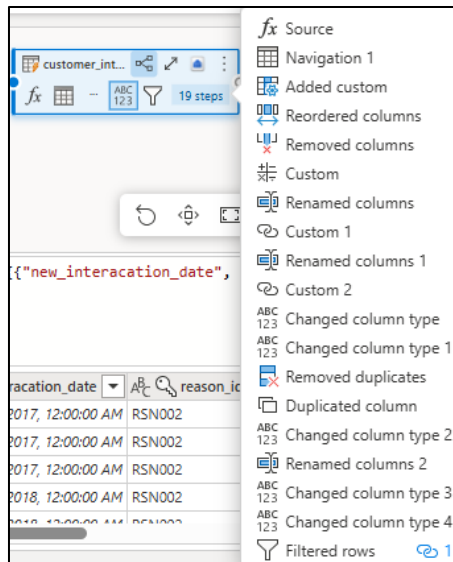
plan_id	abc string	→	bk_plan	ANY varchar
product_name	abc string	→	product_name	ANY varchar
offer_category	abc string	→	offer_category	ANY varchar
offer_type	abc string	→	offer_type	ANY varchar
monthly_price	1.2 double	→	monthly_price	1.2f float
data_allowance	abc string	→	data_allowance	ANY varchar
internet_speed	1.2 double	→	internet_speed	1.2f float
description	abc string	→	description	ANY varchar

6.3. Loading Fact tables

6.3.1. Stg_fact_customer_interactions_cases



The table was constructed using a DataFlow Gen2 and consists of three queries. However, this section will focus solely on describing the **customer_interactions_cases.csv** query, as the other queries are part of the incremental load approach for the fact table and will be explained in more detail later.



There are a total of nineteen steps in the query. However, the most relevant step is the addition of a custom column, **new_interaction_date**, to correct the format of the date, which, similar to the **stg_dim_agent** table, was originally displayed incorrectly.

New column name *

new_interaction_date

Custom column formula * ⓘ

```

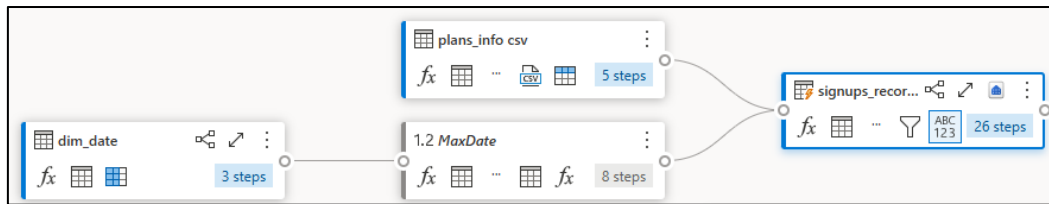
= let
    Digits = Text.PadStart(Text.From
([interaction_date]), 8, "0"),
    Day = Text.Middle(Digits, 0, 2),
    Month = Text.Middle(Digits, 2, 2),
    Year = Text.Middle(Digits, 4, 4)
in
    Date.FromText(Year & "-" & Month & "-" & Day)

```

This code fixes the date format by first ensuring that the **interaction_date** is a string of 8 characters, padding it with leading zeros if necessary. It then extracts the **Day**, **Month**, and **Year** from the string using the **Text.Middle** function. The **Day** is taken from the first two characters, the **Month** from the next two characters, and the **Year** from the last four characters. Finally, the extracted values are combined in the format "YYYY-MM-DD" and converted into a proper date using the **Date.FromText** function.

Moreover, it is worth mentioning that duplicate rows were found in the query, prompting the application of the **remove duplicates** transformation function. This ensured the data's integrity and consistency, contributing to the accuracy of the result.

6.3.2. Stg_fact_signups_record



The **stg_fact_signups_record** table was built using a DataFlow Gen2 and includes four queries. For the purposes of this section, we will focus on the **signups_records** and **plans_info** queries, as the other queries are part of the incremental load process for the fact table and will be covered in more detail later.

The **signups_record** fact table was created by extracting data from two CSV files: **plans_info** and **signups_record**, with the first row used as column headers and standardized column types as needed.

Merge ?

Select a table and matching columns to create a merged table.

signups_record csv

customer_id	plan_id	signup_date	termination_date	1.2 percentage_discount
C354316	PLN002	1/1/2017	null	0
C380045	PLN005	1/1/2017	1/1/2018	0
C145069	PLN006	2/1/2017	2/1/2018	0
C252920	PLN002	2/1/2017	null	0
C329347	PLN004	2/1/2017	2/1/2018	0

Right table for merge *

plans_info csv

data_allowance	internet_speed	description	plan_id
		Entry-level prepaid plan with limited data	PLN001
		High-end prepaid plan with more data	PLN002
		Unlimited data for unlimited use	PLN003

✓ The selection matches 7,796 of 7,796 rows from the first table

Next, we merged the **plans_info** dataset into the **signups_record** dataset using **plan_id** as the common key, performing a left outer join to preserve all rows from **signups_record** and include only matching rows from **plans_info**. The **monthly_price** column from the **plans_info** dataset was expanded to make it available for analysis.

New column name *

Subtraction

Custom column formula * ⓘ

= Duration.Days([termination_date] - [signup_date])

Table.TransformColumns("#Renamed columns", {{"time_difference", each _ / 30, type number}})

id	plan_id	signup_date	termination_date	1.2 percentage_discount	1.2 monthly_price	1.2 time_difference
	PLN002	1/1/2017	null	0	30	null
	PLN002	2/1/2017	null	0	30	null
	PLN002	3/1/2017	null	0	30	null
	PLN005	1/1/2017	1/1/2018	0	80	12.16666667
	PLN006	2/1/2017	2/1/2018	0	150	12.16666667
	PLN006	2/1/2017	2/1/2018	0	150	12.16666667
	PLN004	2/1/2017	2/1/2018	0	50	12.16666667

On the other hand, a new column, **time_difference**, was added to calculate the duration between the **signup_date** and **termination_date**, which was then divided by 30 and rounded to represent the time difference in months. Given the business rule that customers that are acquired prepaid plans do not have termination dates, null values resulted out of this calculation. **These null values were replaced by 1.**

New column name *

Custom column formula * ⓘ

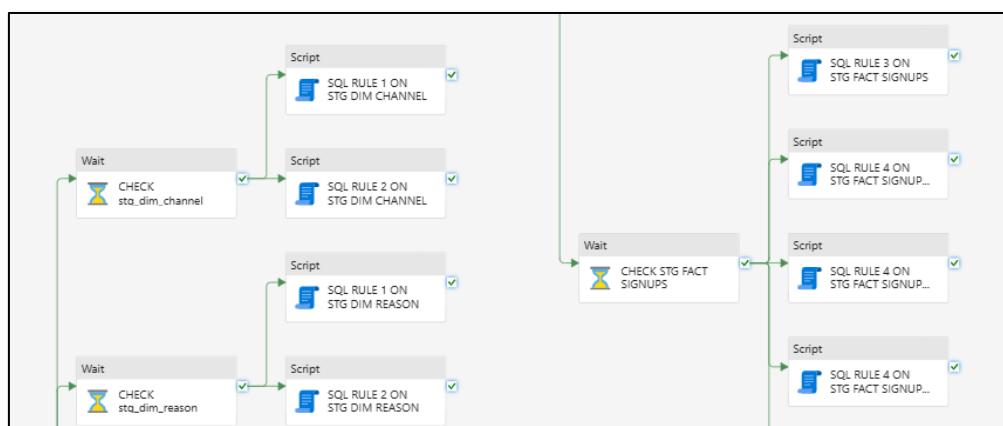
= ([monthly_price] * [time_difference]) * (1-[percentage_discount])

Next, a new column was added to calculate the final price of the product during the contracted months by multiplying the **monthly_price** by the **time_difference** and applying the **percentage_discount**, ensuring the values were converted to numeric type for consistency and further processing.

termination_date	termination_date
null	12/31/2023
null	12/31/2023
null	12/31/2023
1/1/2018	1/1/2018
2/1/2018	2/1/2018
2/1/2018	2/1/2018
2/1/2018	2/1/2018

It is important to add that, for the **termination_date** column, any null values were replaced with **2023-12-31**, the maximum date on the dim_date table.

6.4. Quality Check Validation



For the quality check validation, the four standard rules were followed:

- Rule # 1 Check Integrity of Business Key (for DIM tables)
- Rule # 2 Check Uniqueness of dimension attributes (for DIM tables)
- Rule # 3 Check Integrity of Fact table's PK (for FACT tables)
- Rule # 4 Check relationship of foreign key in Fact table (for FACT tables)

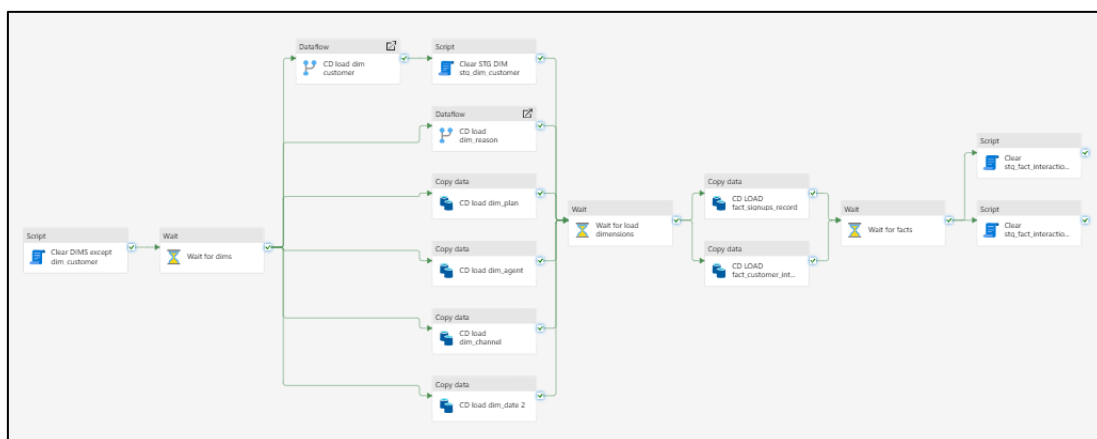
Also, the '**log_quality_checks**' table was created inside the staging area to record any quality issue that could be found after running the validation pipeline.

	123	id_check	ABC	etl_phase	ABC	etl_table	ABC	etl_checktype	ABC	description_result	ABC	etl_result
	1	3		Staging Area		stg_fact_customer_i...		check integrity of fa...		number of rows wit...		OK
	2	2		Staging Area		stg_dim_plan		check uniqueness of...		number of rows NO...		OK
	3	2		Staging Area		stg_dim_agent		check uniqueness of...		number of rows NO...		OK
	4	2		Staging Area		stg_dim_channel		check uniqueness of...		number of rows NO...		OK
	5	4		Staging Area		stg_fact_customer_i...		check parent of FK f...		number of rows wit...		OK
	6	4		Staging Area		stg_fact_signups_rec...		check parent of FK f...		number of rows wit...		OK
	7	2		Staging Area		stg_dim_reason		check uniqueness of...		number of rows NO...		OK
	8	4		Staging Area		stg_fact_customer_i...		check parent of FK f...		number of rows wit...		OK
	9	4		Staging Area		stg_fact_signups_rec...		check parent of FK f...		number of rows wit...		OK
	10	4		Staging Area		stg_fact_signups_rec...		check parent of FK f...		number of rows wit...		OK
	11	3		Staging Area		stg_fact_signups_rec...		check integrity of fa...		number of rows wit...		OK
	12	1		Staging Area		stg_dim_channel		check integrity of BK		number of rows wit...		OK
	13	4		Staging Area		stg_fact_customer_i...		check parent of FK f...		number of rows wit...		OK

<pre>SELECT * FROM log_quality_checks WHERE etl_result = 'FAIL';</pre>											
Messages	Results	Save as table	Open in Excel	...	Copy	Search					
123	id_check	ABC	etl_phase	ABC	etl_table	ABC	etl_checktype	ABC	description_result	ABC	etl_result

As seen, no errors were detected after running the validation pipeline.

6.5. Data Loading Process into the Data Warehouse



Above is the flow of the pipeline that loads data into the tables of the final data warehouse. Due to the previously mentioned mixed approach of full and incremental loads, some modifications were made to the standard pipeline used in the laboratories. Further details on this will be provided in the Extra ETL section.

The pipeline begins by clearing the existing data from the DIM tables, except for **dim_customer**, which follows its own logic and has a dedicated DataFlow Gen2 for loading data into the DW. Once the DIM tables are successfully loaded, two **CopyData** activities are used to load the FACT tables. Finally, two scripts are in place to clear data from the fact tables in the STAGING AREA, which is part of the incremental load process. Note that there are no clearing scripts for the fact tables in the final data warehouse, as only new data is appended to these tables.

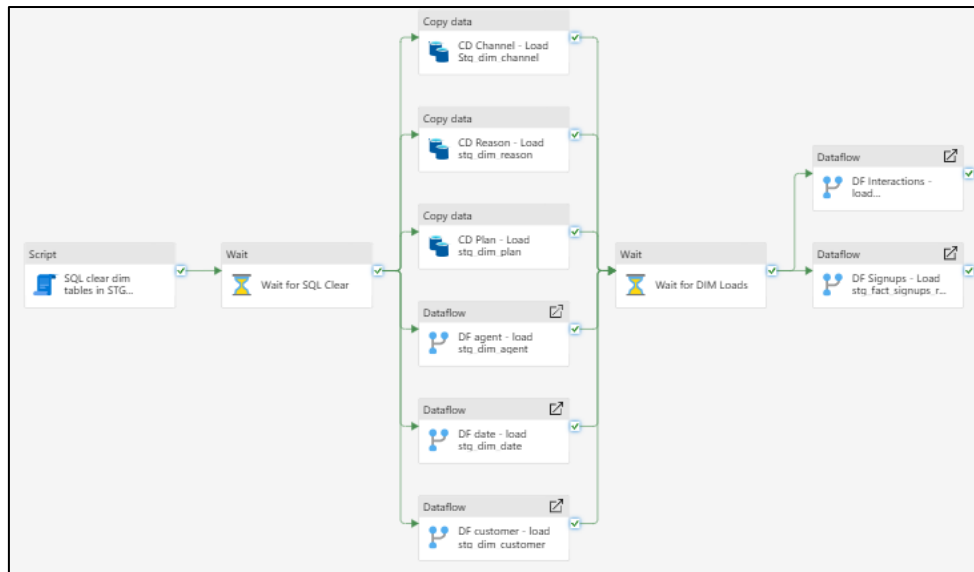
7. Extra ETL work

7.1. Incremental load

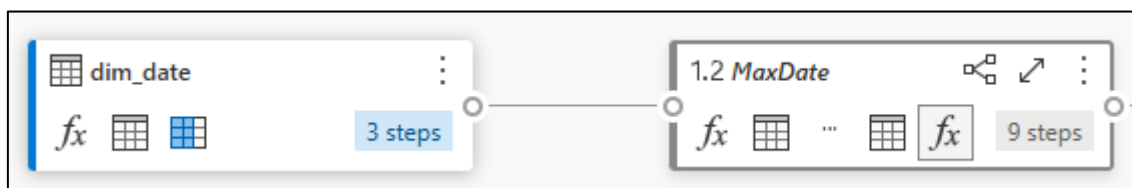
7.1.1. Changes to the staging area pipeline and dataflow

An incremental load approach was implemented for the two fact tables in the pipeline to efficiently manage data updates. This approach ensures that only new or modified records are processed, reducing processing time and resource consumption. The incremental load method complements the full load approach used for the DIM tables, optimizing the overall ETL process.

For simplicity, the process will be explained using the **fact_customer_interactions** table as an example. However, the same steps and methodology were applied to the **fact_signups_records** table, ensuring consistency across both fact tables in the pipeline. This approach was made possible by following the tutorial available at the [Microsoft Learn platform](#), which provided detailed guidance on setting up and implementing the incremental load process using **DataFlows Gen2**.



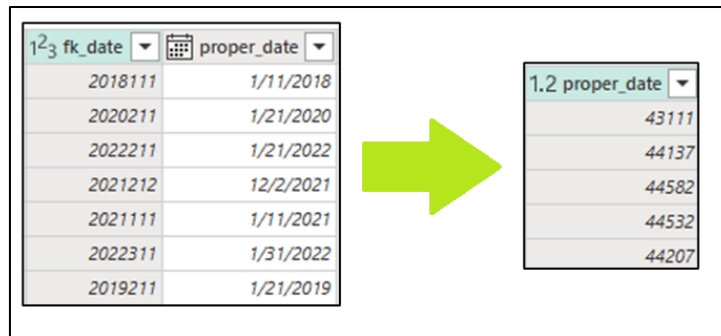
As previously stated, on the staging area pipeline, there are no scripts for clearing the fact tables, as these are set to only load data that has not been previously uploaded into the final data warehouse.



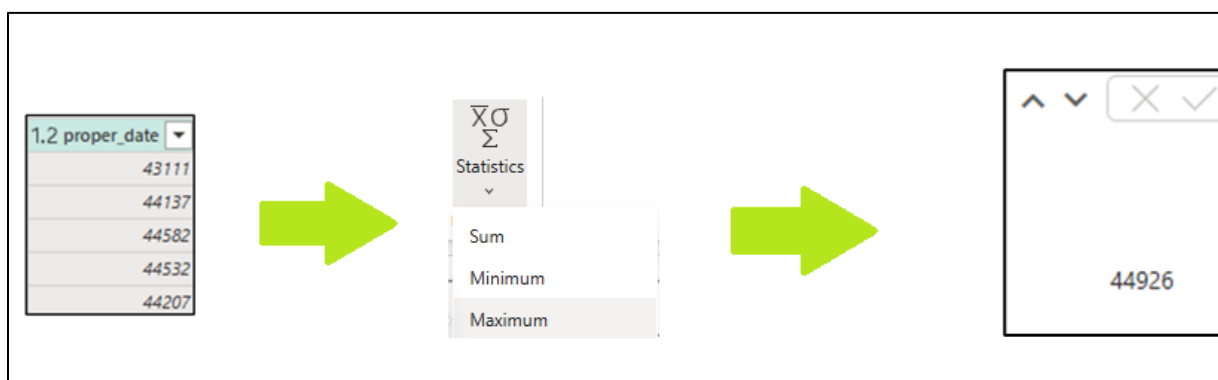
Inside the correspondent dataflow there are two queries used for filtering the new incoming data, 'dim_date', which is the dim table coming directly from the final data warehouse, and the parameter 'MaxDate', which is basically a calculation on the maximum uploaded date for the fact_customers_interactions table on the final data warehouse.

fk_date	proper_date
2018111	1/11/2018
2020211	1/21/2020
2022211	1/21/2022
2021212	12/2/2021
2021111	1/11/2021
2022311	1/31/2022
2019211	1/21/2019

The dim_date query was used to bring the 'proper_date' column to the query of MaxDate (fact_customers_interactions from DW) as there are not foreign keys linked to the original business keys from the date dimension table.



This step was taken to enable a numerical comparison with the **fk_date** column in the fact table in the staging area. By converting the column to a decimal number type, we ensured that only data with a date later than the maximum date in the fact table in the data warehouse would be uploaded.



The next step involves converting the numerical column into a list using the **drill down** function. Afterward, the **Statistics** tool in Power Query is used to identify the maximum value in the list, which corresponds to the latest date in the dataset.

1.2 lastdate_comparison
42890
43023
43036
43144
43445

Once the MaxDate parameter has been set, on the main query inside the dataflow, the 'new_interactions_date' (which is the source for the fk_date column in the DW table) is duplicated, converted to numerical type, and renamed as 'lastdate_comparison'.

Filter rows ?

Apply one or more filter conditions to the rows in this table.

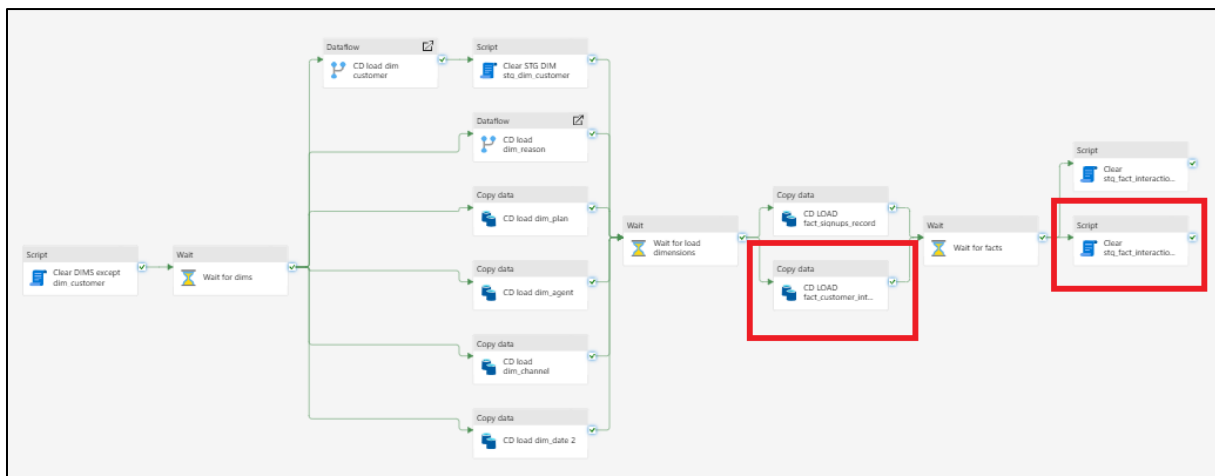
☒ Basic
☐ Advanced

Keep rows where "lastdate_comparison"

Finally, a filter is set so that 'lastdate_comparison' is greater than the MaxDate parameter. Otherwise, no records will be displayed on the query.

This method guarantees that only records with a date later than the last update date stored in the data warehouse are included in the upload, ensuring data consistency and accuracy during the incremental load process.

7.1.2. Modifications to the Data Warehouse Loading Pipeline



Moving on to the pipeline for uploading information into the data warehouse, there are only two containers for the fact table: a **CopyData** activity that loads data from the staging area into the data warehouse, and a script to clear the data from the corresponding fact table in the staging area.

```

IF EXISTS (SELECT 1 FROM
stg_fact_customer_interactions_cases)

BEGIN

    TRUNCATE TABLE stg_fact_customer_interactions_cases;

END

```

The script that clears the data in the staging area is essential to maintain the integrity and accuracy of the incremental load process. By removing the previous data from the fact table in the staging area, we ensure that only the most up-to-date information is loaded into the data warehouse. This step helps prevent redundant data from being processed and ensures the pipeline's efficiency.

7.2. Slowly changing dimension

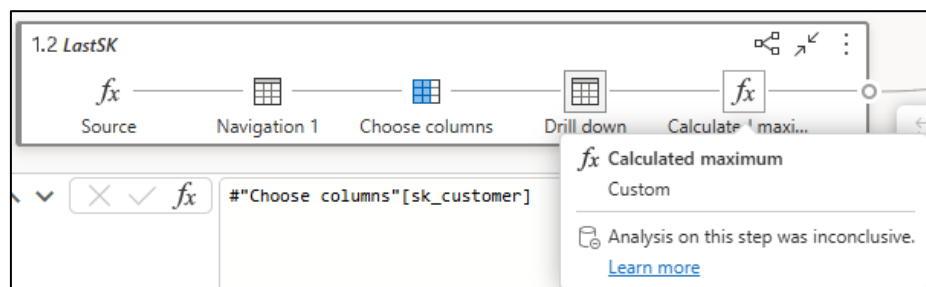
We decided to implement a Slowly Changing Dimension (SCD) Type 2 on the **dim_customer** table to effectively track and store historical changes in customer information. SCD Type 2 allows us to maintain multiple records for a single customer when changes occur, preserving the history of these changes over time.

In this approach, we worked with two DataFlows, each serving a distinct purpose:

The first DataFlow, part of the staging pipeline, filters only the rows that have not yet been uploaded into the data warehouse. This ensures that we only process new or updated records. We used the same methodology explained in the incremental load section, so there is no need for further elaboration on this process.



The second DataFlow, responsible for uploading data into the data warehouse, performs two main tasks: it adds the corresponding surrogate key based on the last available surrogate key in the data warehouse, and it records the exact date and time of the upload by placing a datetime value in the relevant column. This helps track when the data was uploaded and maintains the historical integrity of the information.



To generate the new surrogate key, a parameter is created in the query. First, the **sk_customer** column is brought from the corresponding table in the data warehouse. Using the **Drill Down** function, we extract the maximum value of this column.

the correct **surrogate key** was selected for each customer by linking the **fk_customer** with the latest update timestamp, ensuring accurate and up-to-date customer records in the fact table.

8. Lessons Learned

Throughout the BI project, our group gained valuable insights into working with new tools and concepts. Learning a new environment such as Fabric was both challenging and rewarding, as it required us to quickly adapt and apply new knowledge to a practical context.

The concepts reviewed during the labs, such as dimensional modeling and ETL processes, became the foundation for building a robust and efficient solution that met the business needs. The project also emphasized the benefits of dimensional modeling, showcasing its power in addressing complex business requirements.

During this project we faced different challenges, one of these challenges was navigating the unfamiliar Fabric environment and translating theoretical knowledge into practical application. Understanding and implementing concepts like incremental load and slowly changing dimensions required significant self-study and exploration beyond the project scope.

Additionally, integrating diverse data sources and maintaining data consistency during ETL processes posed technical hurdles that demanded creative problem-solving. Balancing these technical demands while meeting project deadlines required strong collaboration and effective time management.

On the other hand, through this project, we gained hands-on experience with ETL processes, dimensional modeling, and data visualization, which deepened our understanding of how BI solves real-world problems. Teaching ourselves advanced concepts like incremental load and slowly changing dimensions pushed us to think critically and personalize our solution.

Finally, this project showed us that BI is about more than just crunching numbers—it's about combining technical skills with a strong understanding of business needs to make smarter decisions. We also realized how crucial it is to keep learning and adapting to new tools and techniques to stay relevant and build solutions that truly make a difference.

9. Conclusion

This project represents a significant step forward for TelecomPlus in addressing its business challenges and unlocking new growth opportunities. By focusing on improving service quality, understanding customer needs, and optimizing resource allocation, the initiative has delivered a clear and well-structured data warehouse that transforms raw data into actionable insights.

With advanced features like incremental data loading and the ability to track historical changes, the system is both efficient and adaptable, ensuring it evolves alongside the company's needs. Leveraging dimensional modeling, the data warehouse aligns perfectly with TelecomPlus's goals, enabling intuitive and impactful analytics. The integration of Microsoft Fabric further enhances the solution by providing a scalable and reliable platform that supports real-time analytics and ensures data accuracy and accessibility.

This personalized approach has equipped TelecomPlus with a powerful tool to analyze performance, improve customer experiences, and make smarter, faster decisions. By supporting proactive decision-making and responding to market trends, the project lays a strong foundation for sustained growth and innovation in a competitive landscape. Ultimately, this initiative empowers TelecomPlus to thrive in the digital age while delivering greater value to its customers and stakeholders.